Adi Ramanarayan

Construction of Symbol Tables

Week 4

230905380 48

Batch A2

Q1. Generation of global and local symbol tables.

Program:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include "../Week3/identifier_functions.h"

#define MAX_ID_LEN 64
#define MAX_SCOPE_LEN 32
#define TABLE_SIZE 101
#define MAX_FUNCTIONS 50

typedef enum {
    TYPE_INT,
    TYPE_CHAR,
    TYPE_FLOAT,
    TYPE_DOUBLE,
    TYPE_VOID,
    TYPE_BOOL,
    TYPE_UNKNOWN
} DataType;

typedef struct symbol {
    char identifier[MAX_ID_LEN];
    DataType type;
    int size;
    struct symbol *next;
} Symbol;

typedef struct SymbolTable {
    Symbol *buckets[TABLE_SIZE];
    struct SymbolTable *parent;
    char scope_name[MAX_SCOPE_LEN];
    int symbol_count;
} SymbolTable;
```

```
typedef struct FunctionEntry {
    char name[MAX_ID_LEN];
    DataType return_type;
    SymbolTable *local_table;
} FunctionEntry;

FunctionEntry function_table[MAX_FUNCTIONS];
int function_count = 0;


int get_type_size(DataType type) {
    switch (type) {
        case TYPE_INT: return 4;
        case TYPE_CHAR: return 1;
        case TYPE_FLOAT: return 4;
        case TYPE_DOUBLE: return 8;
        case TYPE_BOOL: return 1;
        default: return 0;
    }
}

const char* type_to_string(DataType type) {
    switch (type) {
        case TYPE_INT: return "int";
        case TYPE_CHAR: return "char";
        case TYPE_FLOAT: return "float";
        case TYPE_DOUBLE: return "double";
        case TYPE_VOID: return "void";
        case TYPE_BOOL: return "bool";
        default: return "unknown";
    }
}

DataType keyword_to_type(const char *kw) {
    if (!strcmp(kw, "int")) return TYPE_INT;
    if (!strcmp(kw, "char")) return TYPE_CHAR;
    if (!strcmp(kw, "float")) return TYPE_FLOAT;
    if (!strcmp(kw, "double")) return TYPE_DOUBLE;
    if (!strcmp(kw, "void")) return TYPE_VOID;
    return TYPE_UNKNOWN;
}


unsigned int hash(const char *str) {
    unsigned int h = 0;
    while (*str) h = (h << 5) + *str++;
    return h % TABLE_SIZE;
}
```

```
SymbolTable* create_table(const char *name, SymbolTable *parent) {
    SymbolTable *t = malloc(sizeof(SymbolTable));
    strcpy(t->scope_name, name);
    t->parent = parent;
    t->symbol_count = 0;
    for (int i = 0; i < TABLE_SIZE; i++)
        t->buckets[i] = NULL;
    return t;
}

Symbol* lookup_symbol(SymbolTable *table, const char *id) {
    while (table) {
        Symbol *temp = table->buckets[hash(id)];
        while (temp) {
            if (!strcmp(temp->identifier, id))
                return temp;
            temp = temp->next;
        }
        table = table->parent;
    }
    return NULL;
}

Symbol* lookup_in_current_scope(SymbolTable *table, const char *id) {
    unsigned int idx = hash(id);
    Symbol *temp = table->buckets[idx];
    while (temp) {
        if (!strcmp(temp->identifier, id))
            return temp;
        temp = temp->next;
    }
    return NULL;
}

void insert_symbol(SymbolTable *table, const char *id, DataType type)
{
    // Check if already exists in current scope
    if (lookup_in_current_scope(table, id)) return;

    Symbol *s = malloc(sizeof(Symbol));
    strcpy(s->identifier, id);
    s->type = type;
    s->size = get_type_size(type);
    unsigned int idx = hash(id);
    s->next = table->buckets[idx];
    table->buckets[idx] = s;
    table->symbol_count++;
```

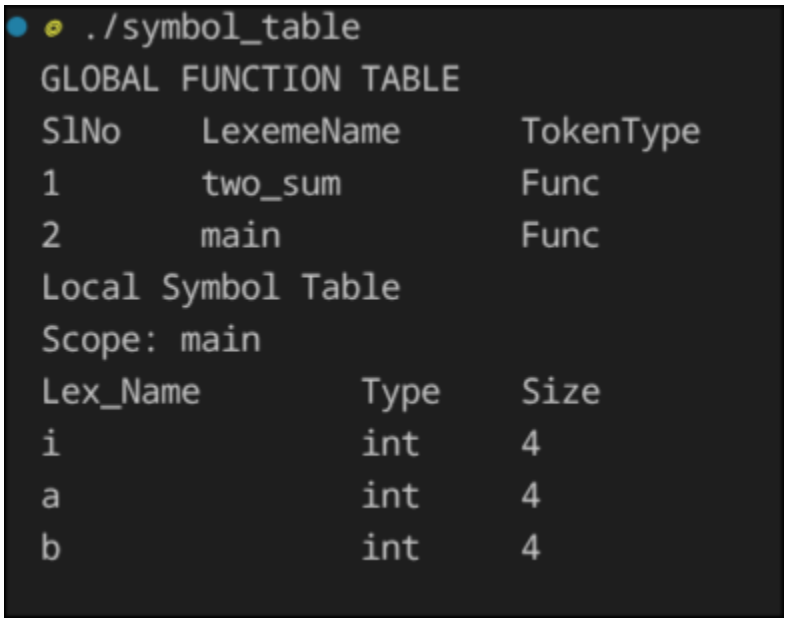
```
}

void add_function(const char *name, DataType return_type, SymbolTable
*local_table) {
    if (function_count >= MAX_FUNCTIONS) return;

    strcpy(function_table[function_count].name, name);
    function_table[function_count].return_type = return_type;
    function_table[function_count].local_table = local_table;
    function_count++;
}

void print_function_table() {
    printf("GLOBAL FUNCTION TABLE\n");
    printf("SlNo\tLexemeName\tTokenType\n");

    for (int i = 0; i < function_count; i++) {
        printf("%d\t%-15s\t%-12s\n",
               i + 1,
               function_table[i].name,
               "Func");
    }
}

void print_local_table(SymbolTable *table) {
    if (!table || table->symbol_count == 0) return;

    printf("Local Symbol Table\n");
    printf("Scope: %s\n", table->scope_name);
    printf("Lex_Name\tType\tSize\n");

    int count = 1;
    for (int i = 0; i < TABLE_SIZE; i++) {
        Symbol *temp = table->buckets[i];
        while (temp) {
            printf("%-15s\t%-7s\t%d\n",
                   temp->identifier,
                   type_to_string(temp->type),
                   temp->size);
            temp = temp->next;
            count++;
        }
    }
}


int main(int argc, char *argv[]) {
    const char *filename = (argc > 1) ? argv[1] : "../Week3/input.c";
```

```
    FILE *src = fopen(filename, "r");
    if (!src) {
        perror("File open error");
        return 1;
    }

    // Initialize lexer state
    PreprocessState st;
    CharBuffer cb;
    initPreprocessState(&st);
    initBuffer(&cb);

    // Parser state
    Token tok;
    DataType current_type = TYPE_UNKNOWN;
    int in_declaration = 0;
    int in_parameter_list = 0;
    int brace_depth = 0;
    int paren_depth = 0;

    char pending_identifiers[10][MAX_ID_LEN];
    int pending_count = 0;

    char current_function[MAX_ID_LEN] = "";
    DataType function_return_type = TYPE_UNKNOWN;
    SymbolTable *current_table = NULL;

    // Main parsing loop
    while ((tok = getNextToken(src, &st, &cb)).type != TOKEN_EOF) {
        // Skip preprocessor directives
        if (tok.type == TOKEN_OPERATOR && tok.lexeme[0] == '#')
            continue;

        // Handle type keywords
        if (tok.type == TOKEN_KEYWORD) {
            DataType t = keyword_to_type(tok.lexeme);
            if (t != TYPE_UNKNOWN) {
                current_type = t;
                in_declaration = 1;
                pending_count = 0;
            }
        }
        // Handle identifiers in declaration context
        else if (tok.type == TOKEN_IDENTIFIER && in_declaration &&
current_type != TYPE_UNKNOWN) {
            // Store identifier for later processing
            strcpy(pending_identifiers[pending_count++], tok.lexeme);
```

```
        }
        // Handle special symbols
        else if (tok.type == TOKEN_SPECIAL_SYMBOL) {
            // Opening parenthesis - might be function declaration
            if (!strcmp(tok.lexeme, "(")) {
                paren_depth++;

                // If we have a pending identifier and we're at
global level (no current table)
                if (pending_count == 1 && !current_table &&
current_type != TYPE_UNKNOWN) {
                    strcpy(current_function, pending_identifiers[0]);
                    function_return_type = current_type;
                    in_parameter_list = 1;
                    pending_count = 0;
                }
            }
            // Closing parenthesis
            else if (!strcmp(tok.lexeme, ")")) {
                paren_depth--;

                if (in_parameter_list && paren_depth == 0) {
                    in_parameter_list = 0;
                    // Parameters have been processed
                    pending_count = 0;
                }

                current_type = TYPE_UNKNOWN;
                in_declaration = 0;
            }
            // Opening brace - function body starts
            else if (!strcmp(tok.lexeme, "{")) {
                brace_depth++;

                // If we just finished a function declaration
                if (strlen(current_function) > 0 && brace_depth == 1)
{
                    // Create local symbol table for this function
                    current_table = create_table(current_function,
NULL);
                    add_function(current_function,
function_return_type, current_table);
                }
            }
            // Closing brace - end of scope
            else if (!strcmp(tok.lexeme, "}")) {
                brace_depth--;
```

```
                // Exiting function body
                if (brace_depth == 0 && current_table) {
                    current_table = NULL;
                    current_function[0] = '\0';
                }
            }
            // Comma - multiple declarations
            else if (!strcmp(tok.lexeme, ",")) {
                // Process the previous identifier(s)
                if (pending_count > 0 && current_type != TYPE_UNKNOWN
&& current_table) {
                    for (int i = 0; i < pending_count; i++) {
                        insert_symbol(current_table,
pending_identifiers[i], current_type);
                    }
                    pending_count = 0;
                }
            }
            // Semicolon - end of declaration
            else if (!strcmp(tok.lexeme, ";")) {
                // Process any pending identifiers
                if (pending_count > 0 && current_type != TYPE_UNKNOWN
&& current_table) {
                    for (int i = 0; i < pending_count; i++) {
                        insert_symbol(current_table,
pending_identifiers[i], current_type);
                    }
                }

                // Reset state
                current_type = TYPE_UNKNOWN;
                in_declaration = 0;
                pending_count = 0;
            }
        }
    }

    // Print results
    print_function_table();

    for (int i = 0; i < function_count; i++) {
        print_local_table(function_table[i].local_table);
    }

    fclose(src);
    return 0;
}
```

Output:


./symbol_table
GLOBAL FUNCTION TABLE
SlNo LexemeName TokenType
1 two_sum Func
2 main Func
Local Symbol Table
Scope: main
Lex_Name Type Size
i int 4
a int 4
b int 4